

PREMIUM CONTRACT: ИСЧЕЗАЮЩИЕ ВЛОЖЕНИЯ (EXPIRING)

Версия: 1.0.0
Дата: 2026-06-14
Статус: ☒ Complete
Доступ:  Только "Шейх"

1. ОБЗОР

1.1 Назначение

Исчезающие вложения (Expiring) — это premium вложение для отправки контента, который автоматически удаляется после прочтения или по истечении таймера. Обеспечивает конфиденциальность и безопасность чувствительной информации.

1.2 Статус доступа

Параметр	Значение
Доступ	 Premium (Шейх)
Категория	Безопасность / Приватность
Сложность	 Средняя
Время реализации	3-4 дня
Шифрование	AES-256-GCM

1.3 Use Cases

СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ

 Конфиденциальные документы
"Отправляю договор – уничтожить после прочтения"

 Временные доступы
"Код от двери действует 2 часа"

 Приватные фото/видео
"Только для тебя, исчезнет через 24 часа"

 Пароли и секреты
"Пароль от WiFi – самоуничтожится"

 Бизнес-переписка
"Коммерческое предложение – не сохранять"

2. ТИПЫ ДАННЫХ

2.1 Основной интерфейс

```
/**
 * Тип вложения
 */
export type ExpiringAttachmentType = 'expiring';

/**
 * Триггеры удаления
 */
export type ExpirationTrigger =
  | 'immediate'      // Сразу после прочтения
  | 'timer'          // По таймеру
  | 'date'           // До конкретной даты
  | 'screenshot'     // После скриншота
  | 'forward'        // После попытки пересылки
  | 'manual';        // Вручную отправителем

/**
 * Уровни защиты
 */
export type ProtectionLevel =
  | 'basic'          // Базовое удаление
  | 'secure'         // + уведомление о скриншоте
  | 'maximum';       // + блокировка пересылки, шифрование

/**
 * Вложение с автоудалением
 */
export interface ExpiringAttachment {
  type: 'expiring';
  attachmentId: string;
  expiringId: string;

  // Оригинальное вложение
  originalType: AttachmentType;
  originalAttachmentId: string;
  originalData: any;

  // Исчезновение
  expiration: ExpirationConfig;
```

```

// Статус
status: ExpiringStatus;
viewedAt?: number;
expiresAt?: number;
destroyedAt?: number;

// Защита
protection: ProtectionConfig;

// Аудит
audit: ExpiringAudit;

// Получатели
recipients: ExpiringRecipient[];

// Отправитель
sender: {
  userId: string;
  displayName: string;
  isSheikh: boolean;
};

// Метаданные
metadata?: Record<string, any>;

createdAt: number;
}

/**
 * Конфигурация исчезновения
 */
export interface ExpirationConfig {
  trigger: ExpirationTrigger;

  // Для timer
  timerSeconds?: number;          // 1 сек - 30 дней

  // Для date
  destroyAtDate?: number;          // timestamp

  // Для immediate
  destroyOnView: boolean;
  destroyOnViewDelay?: number;    // Задержка после просмотра (мс)

  // Для screenshot
  destroyOnScreenshot: boolean;

  // Для forward
  destroyOnForwardAttempt: boolean;

  // Уведомления
  notifyBeforeDestroy: number;    // секунд до удаления

```

```

    notifySenderOnView: boolean;
    notifySenderOnDestroy: boolean;
}

/**
 * Статусы вложения
 */
export type ExpiringStatus =
    | 'active'           // Активно, ожидает просмотра
    | 'viewed'          // Просмотрено, ожидает удаления
    | 'expiring'         // Таймер запущен
    | 'destroyed'        // Уничтожено
    | 'expired'          // Истекло по таймеру
    | 'revoked';         // Отзывано отправителем

/**
 * Конфигурация защиты
 */
export interface ProtectionConfig {
    level: ProtectionLevel;

    // Шифрование
    encryption: {
        enabled: boolean;
        algorithm: 'AES-256-GCM';
        keyRotation: boolean;
    };

    // Скриншоты
    screenshotDetection: {
        enabled: boolean;
        notifySender: boolean;
        blurOnScreenshot: boolean;
        destroyOnScreenshot: boolean;
    };

    // Пересылка
    forwarding: {
        allowed: boolean;
        notifySender: boolean;
        destroyOnForward: boolean;
    };

    // Скачивание
    download: {
        allowed: boolean;
        watermark: boolean;
        watermarkText?: string;
    };

    // DRM
    drm: {
        enabled: boolean;
    };
}

```

```

        preventRecording: boolean;
        secureRenderer: boolean;
    };
}

/**
 * Аудит доступа
 */
export interface ExpiringAudit {
    viewCount: number;
    views: ExpiringView[];

    screenshotCount: number;
    screenshots: ExpiringScreenshot[];

    forwardAttempts: number;
    forwards: ExpiringForwardAttempt[];

    downloadCount: number;
    downloads: ExpiringDownload[];
}

export interface ExpiringView {
    viewerId: string;
    viewerName: string;
    viewedAt: number;
    duration: number; // Как долго смотрел (мс)
    deviceType: 'mobile' | 'desktop' | 'web';
    ipAddress?: string;
    location?: string;
}

export interface ExpiringScreenshot {
    userId: string;
    detectedAt: number;
    deviceType: 'mobile' | 'desktop';
    blocked: boolean;
}

export interface ExpiringForwardAttempt {
    userId: string;
    attemptedAt: number;
    targetChat?: string;
    blocked: boolean;
    reason?: string;
}

export interface ExpiringDownload {
    userId: string;
    downloadedAt: number;
    blocked: boolean;
}

```

```

/**
 * Получатель
 */
export interface ExpiringRecipient {
  userId: string;
  displayName: string;
  status: 'pending' | 'viewed' | 'destroyed';
  viewedAt?: number;
  expiresAt?: number;
  notified: boolean;
}

```

2.2 Обёртки для типов вложений

```

/**
 * Исчезающее изображение
 */
export interface ExpiringImageAttachment extends ExpiringAttachment {
  originalType: 'image';
  originalData: ImageAttachment;
}

/**
 * Исчезающее видео
 */
export interface ExpiringVideoAttachment extends ExpiringAttachment {
  originalType: 'video';
  originalData: VideoAttachment;
}

/**
 * Исчезающее сообщение
 */
export interface ExpiringMessageAttachment extends ExpiringAttachment {
  originalType: 'text';
  originalData: {
    text: string;
    formattedText?: string;
  };
}

/**
 * Исчезающий документ
 */
export interface ExpiringDocumentAttachment extends ExpiringAttachment {
  originalType: 'file';
  originalData: FileAttachment;
}

```

3. API SPECIFICATION

3.1 Endpoints

EXPIRING API ENDPOINTS		
POST	/api/v1/expiring/create	Создать
POST	/api/v1/expiring/:id/view	Пометить прочит.
POST	/api/v1/expiring/:id/revoke	Отозвать
GET	/api/v1/expiring/:id	Получить
GET	/api/v1/expiring/:id/audit	Аудит
GET	/api/v1/expiring/list	Список
DELETE	/api/v1/expiring/:id/destroy	Уничтожить

3.2 Создать исчезающее вложение

Endpoint: POST /api/v1/expiring/create

Headers:

```
Authorization: Bearer <token>
Content-Type: application/json
X-Sheikh-Status: true
```

Request Body:

```
{
  "originalType": "image",
  "originalAttachmentId": "att_123",
  "expiration": {
    "trigger": "timer",
    "timerSeconds": 86400,
    "notifyBeforeDestroy": 3600,
    "notifySenderOnView": true,
    "notifySenderOnDestroy": true
  },
  "protection": {
    "level": "maximum",
    "screenshotDetection": {
      "enabled": true,
      "notifySender": true,
      "blurOnScreenshot": true,
      "destroyOnScreenshot": false
    }
  }
}
```

```

    },
    "forwarding": {
      "allowed": false,
      "notifySender": true,
      "destroyOnForward": true
    },
    "download": {
      "allowed": false,
      "watermark": false
    }
  },
  "recipients": ["user_456", "user_789"]
}

```

Response (Success 201):

```

{
  "success": true,
  "data": {
    "expiringId": "exp_abc123",
    "attachmentId": "att_xyz789",
    "status": "active",
    "expiresAt": 1718486400000,
    "recipients": [
      {
        "userId": "user_456",
        "status": "pending"
      },
      {
        "userId": "user_789",
        "status": "pending"
      }
    ],
    "message": "Вложение создано. Исчезнет через 24 часа после просмотра."
  }
}

```

Response (Premium Required 403):

```

{
  "success": false,
  "error": {
    "code": "PREMIUM_REQUIRED",
    "message": "Исчезающие вложения доступны только пользователям с статусом  
Шейх",
    "upgradeUrl": "/api/v1/premium/upgrade",
    "benefits": [
      "Автоудаление после прочтения",

```

```

        "Защита от скриншотов",
        "Блокировка пересылки",
        "Аудит доступа",
        "Шифрование AES-256"
    ]
}
}

```

3.3 Просмотр вложения

Endpoint: `POST /api/v1/expiring/:id/view`

Request Body:

```

{
  "recipientId": "user_456",
  "viewDuration": 5000
}

```

Response:

```

{
  "success": true,
  "data": {
    "expiringId": "exp_abc123",
    "status": "viewed",
    "viewedAt": 1718400000000,
    "expiresAt": 1718400005000,
    "destroyIn": 5000,
    "content": {
      "type": "image",
      "url": "https://secure.balloo.app/expiring/exp_abc123/content",
      "expiresIn": 5000
    },
    "warning": "Это вложение будет уничтожено через 5 секунд"
  }
}

```

WebSocket Event (отправителю):

```

{
  "event": "expiring_viewed",
  "data": {
    "expiringId": "exp_abc123",
    "viewerId": "user_456",
    "viewerName": "Иван Петров",

```

```
"viewedAt": 1718400000000,  
"deviceType": "mobile",  
"location": "Москва, Россия"  
}  
}
```

3.4 Отозвать вложение

Endpoint: POST /api/v1/expiring/:id/revoke

Request Body:

```
{  
  "reason": "Отправляю правильную версию"  
}
```

Response:

```
{  
  "success": true,  
  "data": {  
    "expiringId": "exp_abc123",  
    "status": "revoked",  
    "revokedAt": 1718400000000,  
    "reason": "Отправляю правильную версию",  
    "recipientsNotified": 2  
  }  
}
```

3.5 Аудит

Endpoint: GET /api/v1/expiring/:id/audit

Response:

```
{  
  "success": true,  
  "data": {  
    "expiringId": "exp_abc123",  
    "viewCount": 2,  
    "views": [  
      {  
        "viewerId": "user_456",  
        "viewerName": "Иван Петров",  
        "viewedAt": 1718400000000,  
        "duration": 5234,  
      }  
    ]  
  }  
}
```

```

    "deviceType": "mobile",
    "location": "Москва, Россия"
  },
  {
    "viewerId": "user_789",
    "viewerName": "Мария Сидорова",
    "viewedAt": 1718400120000,
    "duration": 3100,
    "deviceType": "desktop",
    "location": "Санкт-Петербург, Россия"
  }
],
"screenshotCount": 0,
"screenshots": [],
"forwardAttempts": 1,
"forwards": [
  {
    "userId": "user_456",
    "attemptedAt": 1718400050000,
    "targetChat": "chat_999",
    "blocked": true,
    "reason": "Forwarding not allowed"
  }
],
"downloadCount": 0,
"downloads": []
}
}

```

4. БИЗНЕС-ЛОГИКА

4.1 Создание вложения

```

async function createExpiringAttachment(
  userId: string,
  originalAttachmentId: string,
  expiration: ExpirationConfig,
  protection: ProtectionConfig
): Promise<ExpiringAttachment> {

  // 1. Проверка статуса Шейх
  const user = await getUserById(userId);
  if (!user.is_sheikh) {
    throw new PremiumRequiredError('expiring');
  }

  // 2. Получение оригинального вложения
  const original = await getAttachment(originalAttachmentId);
  if (!original) {

```

```

    throw new AttachmentNotFoundError(originalAttachmentId);
}

// 3. Валидация конфигурации
validateExpirationConfig(expiration);
validateProtectionConfig(protection);

// 4. Шифрование контента (если maximum protection)
let encryptedData: EncryptedData | null = null;
if (protection.level === 'maximum' && protection.encryption.enabled) {
    encryptedData = await encryptAttachment(original);
}

// 5. Создание записи
const expiring: ExpiringAttachment = {
    type: 'expiring',
    attachmentId: generateAttachmentId(),
    expiringId: generateExpiringId(),
    originalType: original.type,
    originalAttachmentId: original.id,
    originalData: original.data,
    expiration,
    status: 'active',
    protection,
    audit: {
        viewCount: 0,
        views: [],
        screenshotCount: 0,
        screenshots: [],
        forwardAttempts: 0,
        forwards: [],
        downloadCount: 0,
        downloads: []
    },
    recipients: [],
    sender: {
        userId: user.id,
        displayName: user.displayName,
        isSheikh: true
    },
    createdAt: Date.now()
};

// 6. Сохранение
await saveExpiringAttachment(expiring);

// 7. Планирование удаления
if (expiration.trigger === 'timer') {
    await scheduleDestruction(
        expiring.expiringId,
        Date.now() + (expiration.timerSeconds! * 1000)
    );
}

```

```
    return expiring;
}
```

4.2 Уничтожение вложения

```
async function destroyExpiringAttachment(
  expiringId: string,
  reason: 'viewed' | 'timer' | 'screenshot' | 'forward' | 'manual'
): Promise<void> {

  const expiring = await getExpiringAttachment(expiringId);
  if (!expiring) return;

  try {
    // 1. Удаление контента
    await deleteAttachmentContent(expiring.originalAttachmentId);

    // 2. Удаление зашифрованных данных
    if (expiring.protection.encryption.enabled) {
      await deleteEncryptionKey(expiringId);
    }

    // 3. Обновление статуса
    expiring.status = 'destroyed';
    expiring.destroyedAt = Date.now();
    await updateExpiringAttachment(expiring);

    // 4. Уведомление отправителя
    if (expiring.expiration.notifySenderOnDestroy) {
      await sendNotification(expiring.sender.userId, {
        type: 'expiring_destroyed',
        expiringId,
        reason,
        destroyedAt: expiring.destroyedAt
      });
    }

    // 5. Уведомление получателей (если ещё не видели)
    for (const recipient of expiring.recipients) {
      if (recipient.status !== 'viewed') {
        await sendNotification(recipient.userId, {
          type: 'expiring_unavailable',
          expiringId,
          reason: 'destroyed_before_view'
        });
      }
    }

    // 6. Логирование
```

```

    await logDestruction({
      expiringId,
      reason,
      timestamp: Date.now()
    });

  } catch (error) {
    logger.error('Failed to destroy expiring attachment', error);
    // Принудительное удаление записи
    await forceDeleteExpiring(expiringId);
  }
}

```

4.3 Детекция скриншотов

```

// Web (Desktop)
async function detectScreenshotWeb(): Promise<boolean> {
  // Visibility API
  document.addEventListener('visibilitychange', () => {
    if (document.visibilityState === 'hidden') {
      // Пользователь мог сделать скриншот
      reportPossibleScreenshot();
    }
  });

  // Print detection
  window.addEventListener('beforeprint', () => {
    reportPossibleScreenshot();
    blurContent();
  });

  return false; // Надёжно детектировать нельзя
}

// Mobile (React Native)
async function detectScreenshotMobile(): Promise<boolean> {
  // iOS/Android имеют события скриншотов
  if (Platform.OS === 'ios') {
    NativeModules.ScreenshotDetector.addListener('onScreenshot', () => {
      handleScreenshotDetected();
    });
  }

  if (Platform.OS === 'android') {
    // Android FLAG_SECURE для блокировки
    NativeModules.AndroidSecurity.setSecureFlag(true);
  }

  return false;
}

```

```

async function handleScreenshotDetected() {
  // 1. Уведомить сервер
  await api.post('/expiring/:id/screenshot', {
    expiringId: currentExpiringId
  });

  // 2. Размыть контент
  blurExpiringContent();

  // 3. Если настроено – удалить
  if (currentProtection.destroyOnScreenshot) {
    await destroyExpiringAttachment(currentExpiringId, 'screenshot');
  }
}

```

5. UI/UX SPECIFICATION

5.1 Отправка исчезающего вложения

 Исчезнувшее вложение
  Только Шейх

Выберите когда уничтожить:

- ☐ Сразу после прочтения
- ☐ Через 5 секунд после прочтения
- ☐ Через 1 минуту после прочтения
- ☐ Через 1 час
- ☐ Через 24 часа
- ☐ Через 7 дней
- ☐ До даты... [📅 25.06.2026]

Защита:

- ☒ Блокировать скриншоты
- ☒ Уведомить о скриншоте
- ☒ Блокировать пересылку
- ☐ Водяной знак

Получатели получают уведомление когда вы просмотрите

[Отмена]
[Отправить 📧]

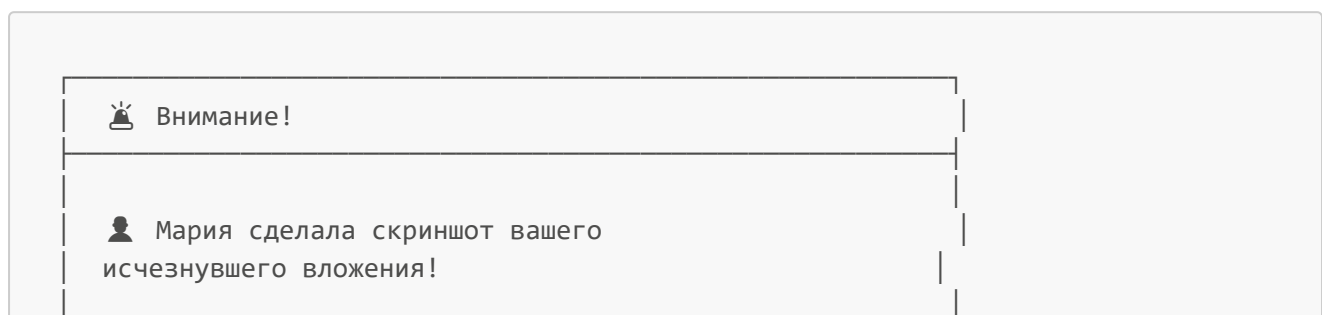
5.2 Просмотр исчезающего вложения



5.3 Уведомление об уничтожении



5.4 Уведомление о скриншоте



📱 Устройство: iPhone 14 Pro
🕒 Время: 15 июня 2026, 14:30
📍 Место: Москва, Россия

Вложение было размыто.

[❌ Игнорировать] [⚠️ Пожаловаться]

6. БЕЗОПАСНОСТЬ

6.1 Шифрование

```
interface EncryptionConfig {
  algorithm: 'AES-256-GCM';
  keySize: 256;
  ivSize: 96;
  tagSize: 128;

  // Key management
  keyStorage: 'hsm' | 'kms' | 'memory';
  keyRotation: boolean;
  rotationInterval: number; // ms

  // Access
  keyDerivation: 'PBKDF2' | 'Argon2';
  iterations: number;
}

async function encryptAttachment(
  attachment: Attachment
): Promise<EncryptedData> {
  // 1. Генерация ключа
  const key = await crypto.subtle.generateKey(
    { name: 'AES-GCM', length: 256 },
    true,
    ['encrypt', 'decrypt']
  );

  // 2. Генерация IV
  const iv = crypto.getRandomValues(new Uint8Array(12));

  // 3. Шифрование контента
  const encoded = new TextEncoder().encode(attachment.data);
  const encrypted = await crypto.subtle.encrypt(
    { name: 'AES-GCM', iv },
    key,
    encoded
  );
}
```

```

);

// 4. Сохранение ключа в KMS
const keyId = await storeKeyInKMS(key);

return {
  encryptedData: Buffer.from(encrypted),
  iv: Buffer.from(iv),
  keyId,
  algorithm: 'AES-256-GCM'
};
}

```

6.2 Secure Renderer

```

// React Native - блокировка скриншотов
import ReactNativeBlobUtil from 'react-native-blob-util';

// Android
ReactNativeBlobUtil.android.disableScreenshot();

// iOS
NativeModules.SecureView.enableSecureMode();

// Web - защита от копирования
function enableSecureRenderer(element: HTMLElement) {
  element.style.userSelect = 'none';
  element.style.webkitUserSelect = 'none';

  // Context menu
  element.addEventListener('contextmenu', (e) => e.preventDefault());

  // Drag
  element.addEventListener('dragstart', (e) => e.preventDefault());

  // Copy
  element.addEventListener('copy', (e) => e.preventDefault());

  // Print
  const style = document.createElement('style');
  style.textContent = '@media print { body { display: none; } }';
  document.head.appendChild(style);
}

```

7. МЕТРИКИ

```
interface ExpiringMetrics {
    // Создание
    totalCreated: number;
    createdToday: number;
    byType: Record<AttachmentType, number>;

    // Просмотры
    totalViews: number;
    averageViewDuration: number;
    viewRate: number; // % просмотренных

    // Уничтожения
    totalDestroyed: number;
    byReason: {
        viewed: number;
        timer: number;
        screenshot: number;
        forward: number;
        manual: number;
    };

    // Безопасность
    screenshotsDetected: number;
    forwardsBlocked: number;
    downloadsBlocked: number;

    // Уведомления
    notificationsSent: number;
    alertRate: number; // % с alert'ами
}
```

 **Статус документа:** Complete

 **Baloo - Проверни общение!**